

「README」

DocGen TW · 台灣法律合約自動產生平台

3 分鐘產出附完整中華民國法條依據的合約，雙方電子簽署留存 IP、時間戳與簽名雜湊，並整合藍新金流與催款 / 到期提醒。

DocGen TW 是台灣場景導向的合約自動化平台，主軸為「範本填寫 → 版本化合約 → 雙方簽署 → 提醒與支付升級」。

專案狀態：

- 架構完整，具備 API、前端、資料模型、外部整合與部署/維運文件。
- 可部署到 Vercel，`DEPLOY.md` 提供生產核對清單。

專案文件

- [docs/ARCHITECTURE.md](#)：系統架構、技術棧、資料模型（含 ER 圖）。
- [docs/FLOWS.md](#)：主要業務流程、關鍵時序與路由映射。
- [docs/PAGES.md](#)：頁面路由、API 端點、授權狀態總覽。
- [docs/OPERATIONS.md](#)：安裝、環境變數、啟動、部署與維運。

快速開始

```
npm install
npm run dev
```

專案簡介

DocGen TW 是一套針對台灣使用情境設計的法律文件自動化 SaaS。使用者從 10 種常用範本（承攬、NDA、借貸、委任、勞動、租賃、買賣、催款通知書、存證信函草稿、自訂空白）逐欄填表即可產出合約，每一條款都附上對應法令引用。合約支援甲乙雙方線上電子簽署、PDF 匯出存證、規則式（可選 AI 增強）風險檢查，以及案件資料夾、付款里程碑與到期 Email 自動提醒。付費解鎖透過藍新金流（NewebPay）結帳，並提供 Bearer Token 形式的公開 API。

免責：本平台提供文件自動化與風險提示，依現行法律一般情形編製，不取代執業律師意見。

核心功能

- 10 種台灣法律範本：定義於 `lib/templates.ts`，每個範本含動態欄位、分組、`{{欄位}}` 樣板填充與逐條法令依據（`legal` 陣列）。
- 動態表單 + 即時預覽：`/contracts/new` 左側填表、右側即時預覽（`components/ContractPreview.tsx`、`Stepper.tsx`），數字自動轉國字大寫（`lib/numberToChinese.ts`）。

- **雙方電子簽署**：手寫簽名板（`components/SignaturePad.tsx`，`react-signature-canvas`），收件方透過簽署連結 `/contracts/[id]/sign?token=...` 簽署；留存簽署時間戳、IP 與 SHA-256 簽名雜湊。合約生命週期 `UNSIGNED` → `SENDER_SIGNED` → `AWAITING_RECIPIENT` → `FULLY_SIGNED`（`lib/services/signing_service.ts`、`lib/contract-store.ts`）。
- **PDF 產出與存證**：`@react-pdf/renderer` 伺服器端渲染（`lib/pdf/render.tsx`），`GET /api/contracts/[id]/pdf` 下載。簽名圖存 Vercel Blob，未設 token 時 dev 下 fallback 至 `public/signatures/`。
- **合約風險檢查**：規則式檢查（`lib/risk-rules.ts`、`risk-rules-text.ts`，含利率上限、永久保密、外國管轄等規則），可選 OpenRouter LLM 增強（`lib/llm-risk.ts` / `openrouter.ts`，預設 `google/gemini-2.5-pro:free`），介面在 `/check`，並可產生可分享連結 `/shared/check/[id]`。
- **案件資料夾 + 里程碑**：`/cases` 管理案件、附件（Case / CaseAttachment），合約可掛 Milestone 付款 / 交付節點（`lib/milestone-parser.ts`、`case-store.ts`）。
- **到期自動提醒**：每日 Cron `GET /api/cron/reminders`（`vercel.json` 排程 `0 1 * * *`）對 D-7 / D-1 / D-0 / D+1 里程碑寄 Email（Mailgun），逾期可一鍵生成催款通知書草稿，並觸發使用者自訂 webhook（Slack / Discord / Make / n8n / Zapier）。
- **金流與訂閱**：藍新金流結帳（`/checkout` → `POST /api/payment/newebpay/checkout`），背景 `notify` 與前景 `return` 回呼，`Order` / `BillingProfile` 記錄付款與方案（FREE / Pro NT\$299 月、Pack NT\$499 / 90 天、單份解鎖 NT\$99）。
- **使用者認證**：Email Magic Link 登入（`POST /api/auth/email/start` → `verify`，15 分鐘一次性連結，cookie uid），無密碼。
- **公開 API**：`POST /api/v1/contracts`，`Authorization: Bearer dk_<key>`，沿用 FREE/PRO 額度與速率限制（`lib/rate-limit.ts`）；API Key 僅存 SHA-256 雜湊（`lib/api-keys.ts`），於 `/settings` 管理。
- **後台**：`/admin`（共享金鑰 `ADMIN_KEY` 認證，`lib/admin-auth.ts`），含合約 / 用量 / 推薦 / webhook 重送與 CSV 匯出（`/api/admin/*`）。
- **雙語 (i18n)**：`zh-TW`（預設）與 `en`（`/en/*`），語系由 `proxy.ts` 注入 `x-pathname` header + `lib/i18n/*` 解析。
- **律師媒合**：免責頁的律師轉介表單與 CTA（`LawyerReferral` 模型、`/api/referrals`）。
- **SEO**：`app/sitemap.ts`、`robots.ts`、`components/JsonLd.tsx`（Organization / SoftwareApplication / WebSite 結構化資料）。

技術棧

類別	使用技術
框架	Next.js 16.2.4 (App Router)、React 19.2.4
語言	TypeScript 5
樣式 / UI	Tailwind CSS v4、shadcn (base-nova)、@base-ui/react、lucide-react、 <code>class-variance-authority</code> 、 <code>tailwind-merge</code> 、 <code>tw-animate-css</code>
資料庫 / ORM	PostgreSQL (Neon)、Prisma 6 + <code>@prisma/client</code>
檔案儲存	Vercel Blob (<code>@vercel/blob</code>)
PDF	<code>@react-pdf/renderer</code>
簽名	<code>react-signature-canvas</code>
金流	NewebPay (藍新金流， <code>lib/payment/newebpay.ts</code> ，自研 HTTP，無 SDK)
Email	Mailgun (<code>lib/mailgun.ts</code> ，自研 HTTP，無 SDK)
AI (選用)	OpenRouter (預設 <code>google/gemini-2.5-pro:free</code>)

類別	使用技術
測試	Vitest 4 (@vitest/coverage-v8)
部署	Vercel (含 Cron)

目錄結構

```
docgen-tw/
├── app/
│   ├── page.tsx           # 首頁 (範本畫廊 / 定價 / 流程)
│   ├── layout.tsx         # 根 layout + metadata + JSON-LD
│   ├── contracts/         # 建立 / 檢視 / 簽署 / 版本
│   ├── cases/             # 案件資料夾
│   ├── check/             # 合約風險檢查
│   ├── checkout/ · payment/ # 結帳與付款結果
│   ├── admin/ · settings/  # 後台 / 帳號設定
│   ├── en/               # 英文版頁面
│   └── api/               # 路由 handlers (contracts / cases / milestones
                           # / payment / auth / billing / admin / cron / v1 ...)
├── components/            # React UI 元件 (SignaturePad、Stepper、TopNav ...)
├── lib/
│   ├── templates.ts       # 10 種合約範本定義 (核心)
│   ├── contract-store.ts · case-store.ts
│   ├── payment/           # newebpay.ts · pricing.ts
│   ├── pdf/render.tsx     # PDF 渲染
│   ├── risk-rules*.ts · llm-risk.ts · openrouter.ts
│   ├── mailgun.ts · notify.ts · webhooks.ts
│   ├── api-keys.ts · billing.ts · rate-limit.ts · admin-auth.ts
│   └── i18n/              # 多語系字典與 locale 解析
├── data/templates/contract_types.json
├── prisma/schema.prisma   # 12 個 model (Contract / Case / Milestone / Order ...)
├── tests/                 # Vitest 單元測試 (newebpay / risk-rules / csv ...)
├── proxy.ts               # 注入 x-pathname header (locale 用)
├── vercel.json             # Cron 排程
├── DEPLOY.md              # 部署 checklist
└── next.config.ts · prisma.config.ts · vitest.config.ts
```

本機開發

需求：Node.js 20+、一個 PostgreSQL 連線字串 (建議 [Neon](#))。

```
# 1. 安裝依賴 (postinstall 會自動跑 prisma generate)
npm install

# 2. 設定環境變數 (見下方)
# 本機可用 Vercel CLI 拉線上變數：
vercel env pull .env.local

# 3. 推送 schema 到資料庫
npm run db:push
```

4. 啟動開發伺服器

```
npm run dev
```

開啟 <http://localhost:3000>。

其他指令：

```
npm run build      # prisma generate + prisma db push + next build
npm run start      # 啟動 production server
npm run lint       # ESLint
npm test           # Vitest (含 NewwebPay TradeSha regression)
npm run db:migrate # prisma migrate deploy
```

多數功能（建立合約、認證、里程碑）在未連資料庫時會回傳 503 或降級，請先設好 DB 連線。OpenRouter / Mailgun / Blob / NewwebPay 未設定時各自 graceful fallback 或回報「未設定」，不會中斷主流程。

環境變數

以實際 `process.env` 引用整理（本專案 `.gitignore` 排除所有 `.env*`，未附 `.env.example`）。

資料庫（必填）

變數	必填	說明
<code>DATABASE_POSTGRES_PRISMA_URL</code>	✓	Prisma 應用查詢用的 pooled 連線（ <code>schema.prisma url</code> ，Vercel + Neon 整合自動注入）
<code>DATABASE_URL_UNPOOLED</code>	✓	migration 用的 direct 連線（ <code>schema.prisma directUrl</code> ）
<code>DATABASE_URL</code>	✓	應用層判斷「DB 是否已設定」的旗標（多處 API gating 與 <code>prisma.config.ts</code> 讀取）；通常與 pooled URL 同值

Vercel 的 Neon 整合會自動注入前兩者；本機請手動補齊（三者可指向同一 Neon DB）。

金流 — NewwebPay 藍新（啟用付費時必填）

變數	必填	說明
<code>NEWEBPAY_MERCHANT_ID</code>	■	商店代號
<code>NEWEBPAY_HASH_KEY</code>	■	HashKey (32 bytes)
<code>NEWEBPAY_HASH_IV</code>	■	HashIV (16 bytes)
<code>NEWEBPAY_API_BASE</code>	■	<code>https://core.newebpay.com</code> (正式) / <code>https://ccore.newebpay.com</code> (測試)

TradeSha 公式為 `SHA256("HashKey={KEY}&{tradeInfoHex}&HashIV={IV}")`，無 `TradeInfo=` 前綴；`tests/newebpay.test.ts` 鎖定此規則。

Email — Mailgun（簽署完成 / 登入連結 / 到期提醒；未設則略過寄信）

變數	必填	說明
MAILGUN_API_KEY	☐	API Key
MAILGUN_DOMAIN	☐	寄件網域
MAILGUN_FROM	☐	寄件者（預設 DocGen TW <noreply@{domain}> ）
MAILGUN_API_BASE	☐	預設 https://api.mailgun.net/v3 （EU 區改 api.eu.mailgun.net ）

檔案儲存 — Vercel Blob

變數	必填	說明
BLOB_READ_WRITE_TOKEN	☐	簽名圖儲存；Vercel 連 Blob Store 後自動注入。未設時 dev 下 fallback 至 public/signatures/

AI 風險檢查（選用）

變數	必填	說明
OPENROUTER_API_KEY	☐	啟用 /check 的 LLM 增強；未設則只跑規則式檢查
OPENROUTER_MODEL	☐	預設 google/gemini-2.5-pro:free
SMART_ROUTER_URL	☐	自架智慧路由端點（ lib/router-client.ts ）

站台 URL / Cron / 後台 / 通知

變數	必填	說明
NEXT_PUBLIC_APP_URL	☐	站台對外 URL（簽署連結、metadata、回呼 origin）。預設 https://docgen-tw.vercel.app
NEXT_PUBLIC_SITE_URL	☐	站台 URL（部分連結 origin 使用）
CRON_SECRET	☐	保護 /api/cron/reminders；未設時該端點開放（僅建議 dev）
ADMIN_KEY	☐	/admin 後台共享金鑰；未設時後台一律拒絕
ADMIN_ALERT_WEBHOOK	☐	系統告警 webhook
WEBHOOK_FAIL_ALERT_THRESHOLD	☐	webhook 連續失敗告警門檻
REFERRAL_NOTIFY_TO	☐	律師媒合表單通知收件信箱

部署（Vercel）

完整步驟見 [DEPLOY.md](#)。摘要：

- 資料庫：在 Vercel 接上 Neon 整合（自動注入 DATABASE_POSTGRES_PRISMA_URL / DATABASE_URL_UNPOOLED），並補上 DATABASE_URL。
- Blob：Vercel → Storage → 建立 Blob Store 並 connect 至專案，BLOB_READ_WRITE_TOKEN 自動注入。

3. **金流 / Email**：於 Project Settings → Environment Variables 填入 NewwebPay 與 Mailgun 變數（去除前後空白）。

4. **Cron**：`vercel.json` 已定義 `/api/cron/reminders` 每日 `0 1 * * *`，建議設 `CRON_SECRET` 保護。

5. **部署**：

```
vercel link  
vercel --prod
```

`build script` 會自動 `prisma generate` + `prisma db push` + `next build`。

部署後驗證清單（範本畫廊、建立合約寫入 DB、簽名圖 URL、結帳導向、雙方簽署達 `FULLY_SIGNED` 等）見 `DEPLOY.md §6`。

授權

Private — All rights reserved.（本倉庫 `private: true`，未附 LICENSE 檔案。）

「ARCHITECTURE.md」

系統架構

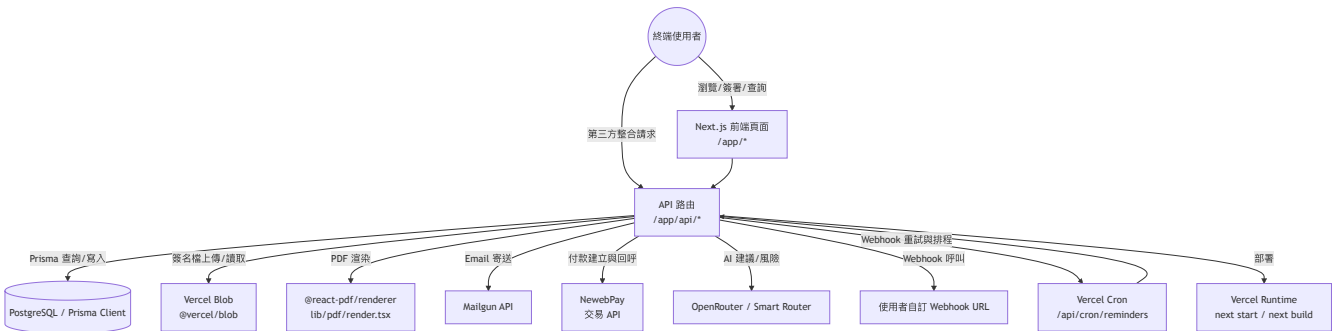
專案總覽

DocGen TW 是以 Next.js App Router 為基礎的台灣法律文件自動化服務，核心目標是：使用者可用表單套用合約範本、送出後產生含法條參考的合約，並支援雙方電子簽署、里程碑提醒、付款升級與 API 簽約串接。服務主要提供給個人接案者、律師團隊與小型商務主體。

技術棧

類別	使用技術	依據
應用框架	<code>next</code> <code>react</code>	<code>package.json dependencies</code> 、 <code>app/</code> 路由模式
語言	<code>TypeScript</code>	<code>package.json devDependencies</code>
ORM / 資料	<code>prisma</code> + <code>@prisma/client</code> 、 <code>PostgreSQL</code>	<code>package.json</code> 、 <code>prisma/schema.prisma</code>
API 風格	Next.js App Router Route Handlers (<code>app/api/**/*.route.ts</code>)	<code>app/api</code> 檔案實作
文件存放	<code>@vercel/blob</code>	<code>lib/signing_service.ts</code> (簽名圖片)
PDF	<code>@react-pdf/renderer</code>	<code>lib/pdf/render.tsx</code>
金流	自研 NewwebPay 串接	<code>lib/payment/newwebpay.ts</code>
Mail	Mailgun HTTP	<code>lib/mailgun.ts</code>
AI/風險檢查 (選用)	OpenRouter、Smart Router、 <code>llm-risk.ts</code>	<code>lib/openrouter.ts</code> 、 <code>lib/router-client.ts</code>
部署	Vercel (含 Cron)	<code>vercel.json</code> 、 <code>README</code> 、 <code>DEPLOY.md</code>
測試	Vitest	<code>package.json scripts</code> 、 <code>tests/</code>

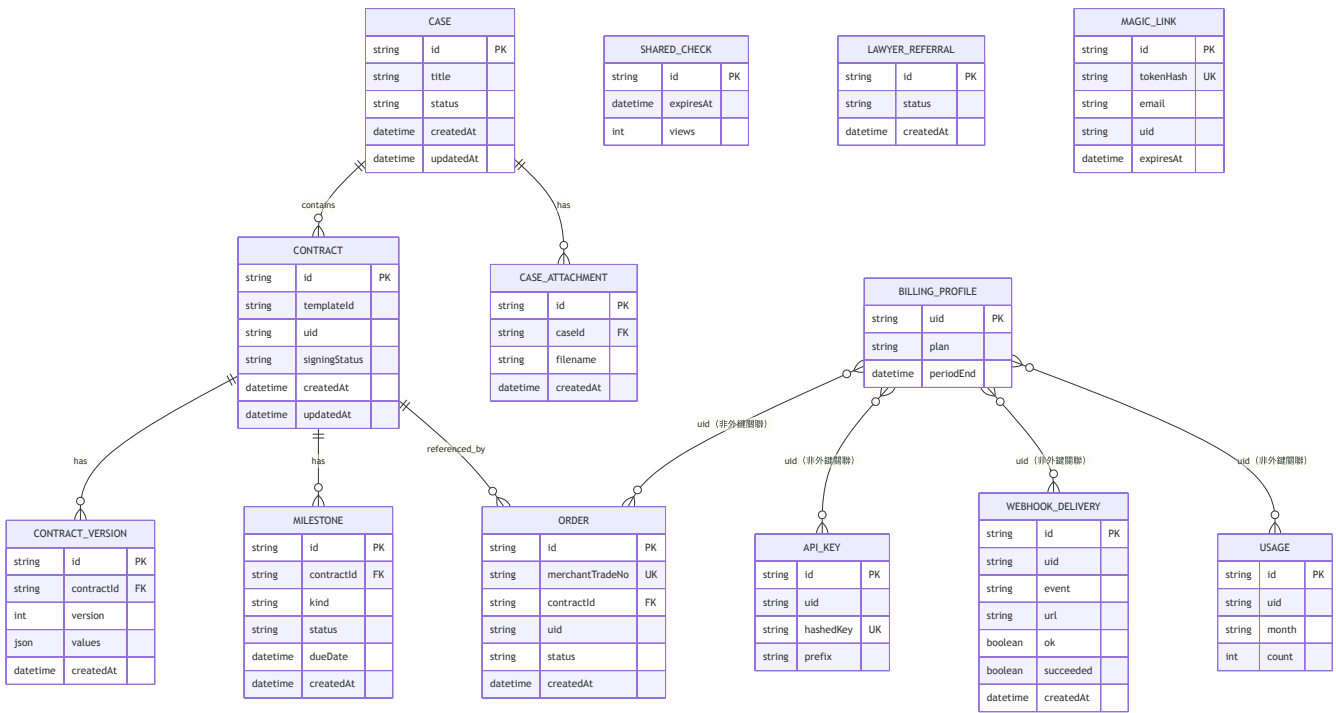
架構圖



主要目錄結構

目錄/檔案	用途
app/	App Router 頁面與 API Route Handlers (<code>app/page.tsx</code> 、 <code>app/api/.../route.ts</code>)
app/api/cron/reminders	每日到期提醒與 webhook 重試入口
app/api/admin/*	管理員後台 API (合約/用量/推薦/Webhook)
app/api/v1/contracts	對外公開 API (API Key)
components/	客戶端 UI 元件 (簽名筆、表單步驟、預覽等)
lib/	核心商務/資料邏輯 (合約、案件、付款、風險、Webhook、Email、JWT 無)
lib/payment/	NewwebPay 參數組裝、簽章與解碼
lib/i18n/	中文/英文字典與路由文案
prisma/schema.prisma	資料模型定義
prisma.config.ts	Prisma 設定 (DATABASE_URL)
DEPLOY.md	Vercel/Neon/支付的部署檢核清單
proxy.ts	locale proxy 與路徑標頭
next.config.ts	Next.js 執行緒與 headers 設定

資料模型 (Prisma)

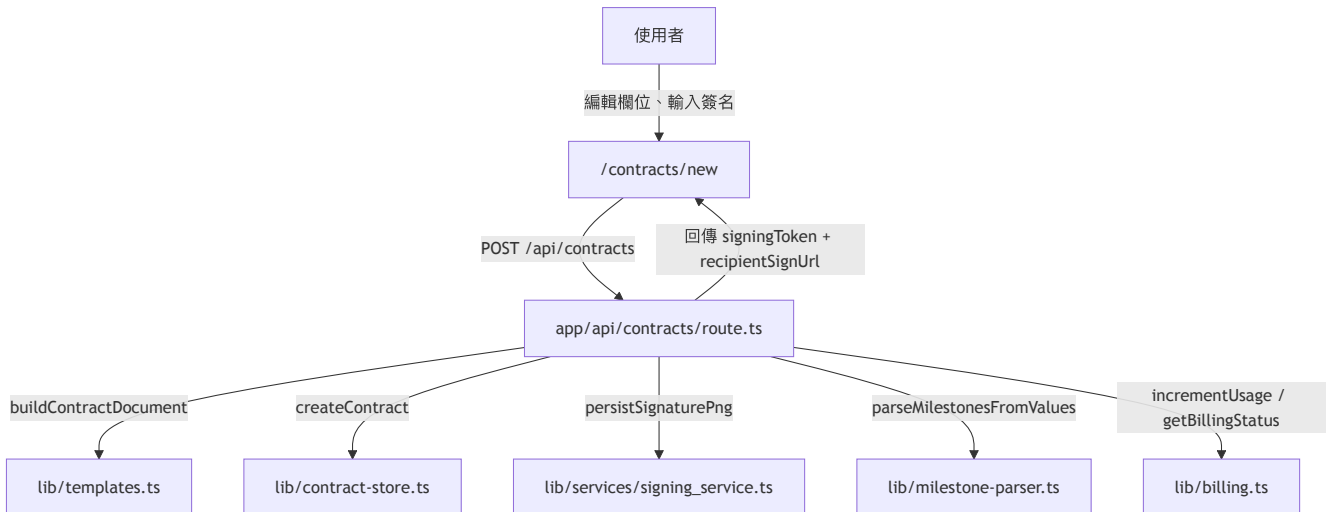


註：部分帳務/權限關聯（例如 uid 對應）是以欄位值串接，未使用 Prisma 明確 @relation。以程式邏輯仍會同時以 uid 進行關聯查詢與保護。

「FLOWS.md」

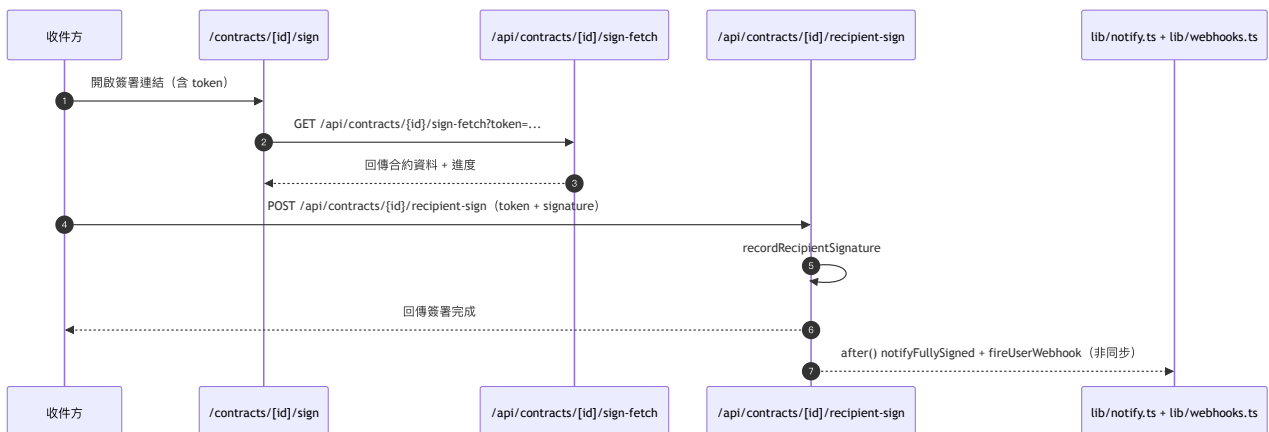
操作與業務流程

1) 建立合約與寄出簽署連結



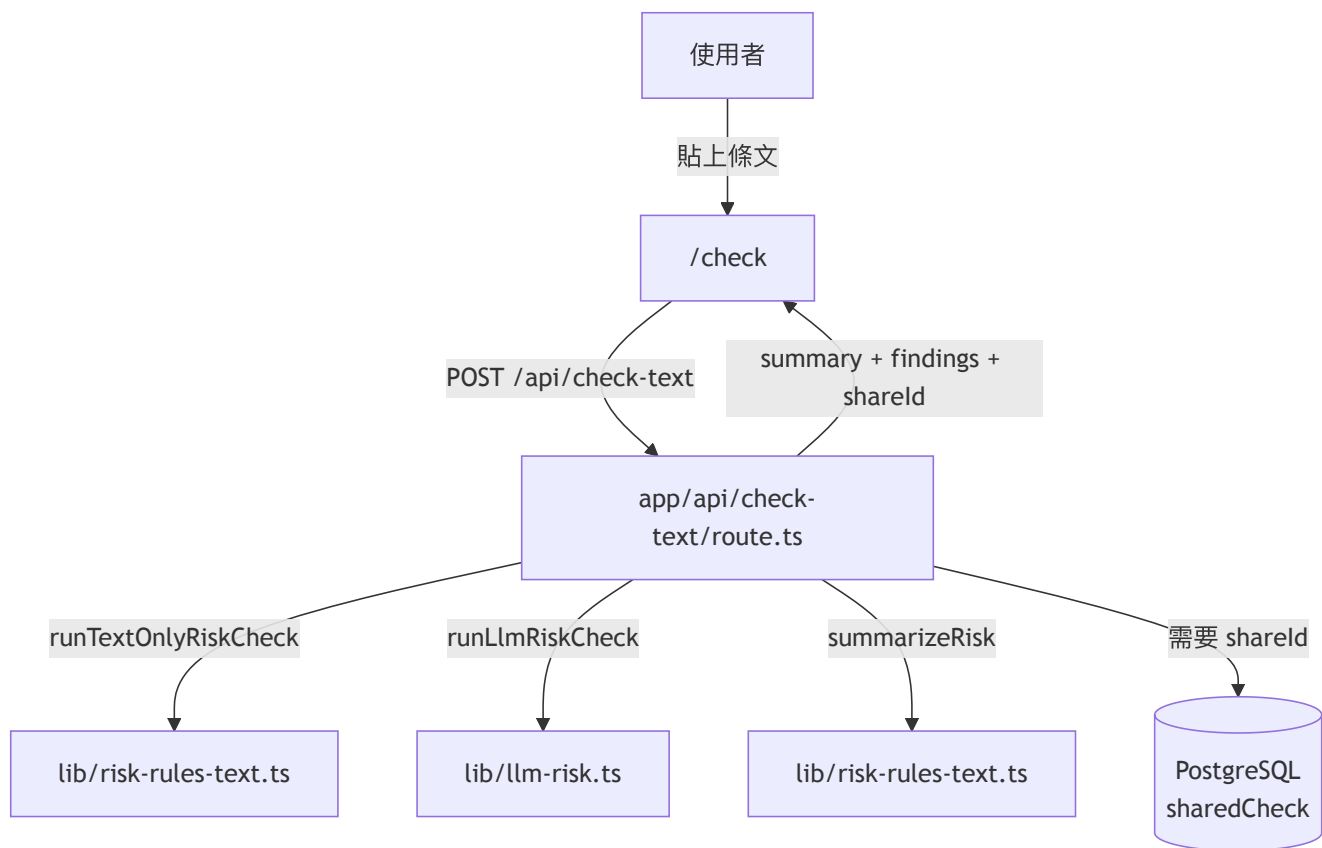
使用者在 `/contracts/new` 產生草稿並送出後，後端先判斷 `uid` 與額度，再建立 `Contract`、簽名圖、可選自動里程碑。回傳資料含 `signingToken` 與簽署連結。

2) 收件方簽署流程



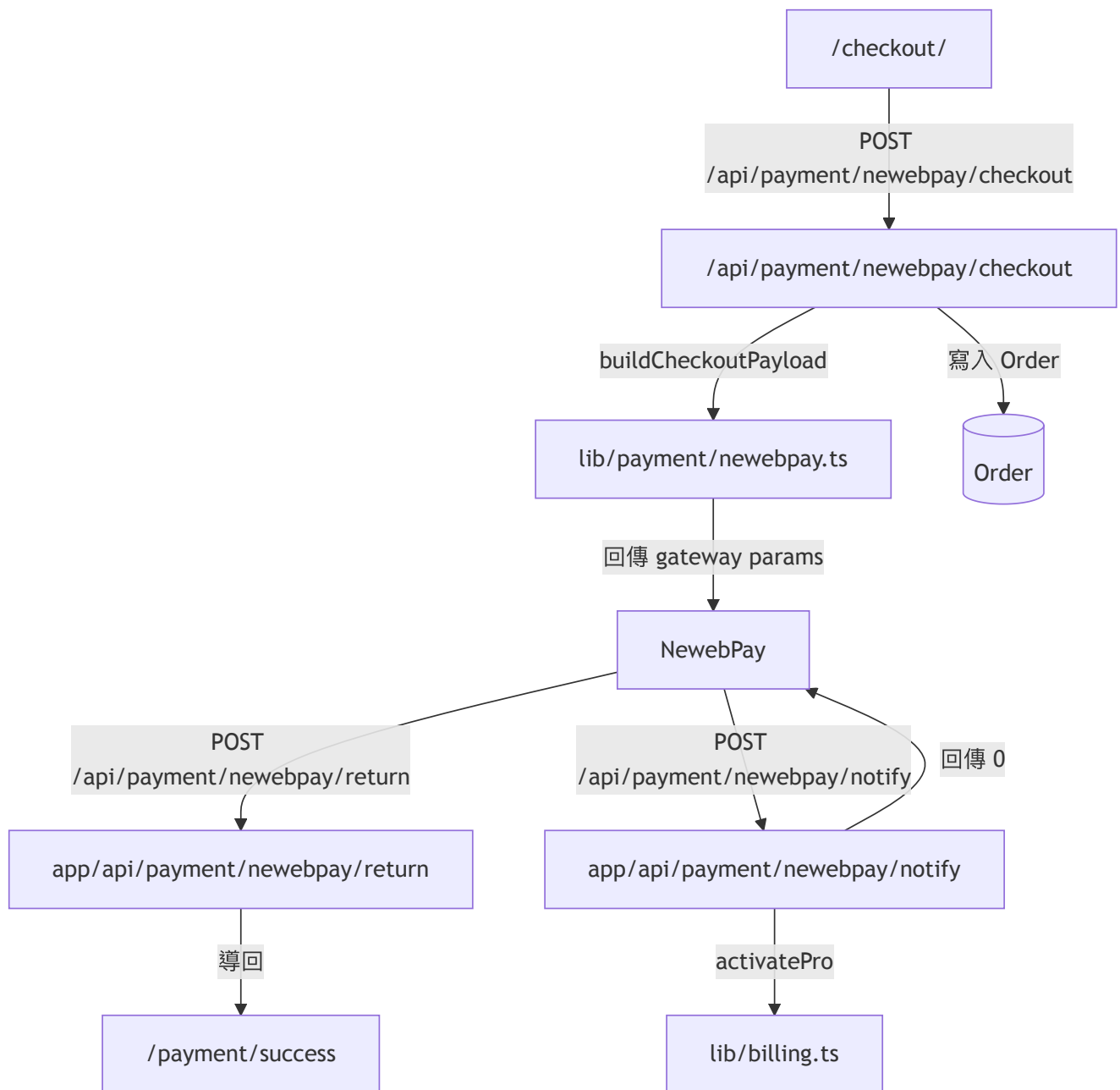
收件方需憑 `token` 進入，不需登入。簽署成功後會更新 `Contract.signingStatus`，並觸發 PDF/郵件通知與 `webhook`。

3) 合約風險檢查流程



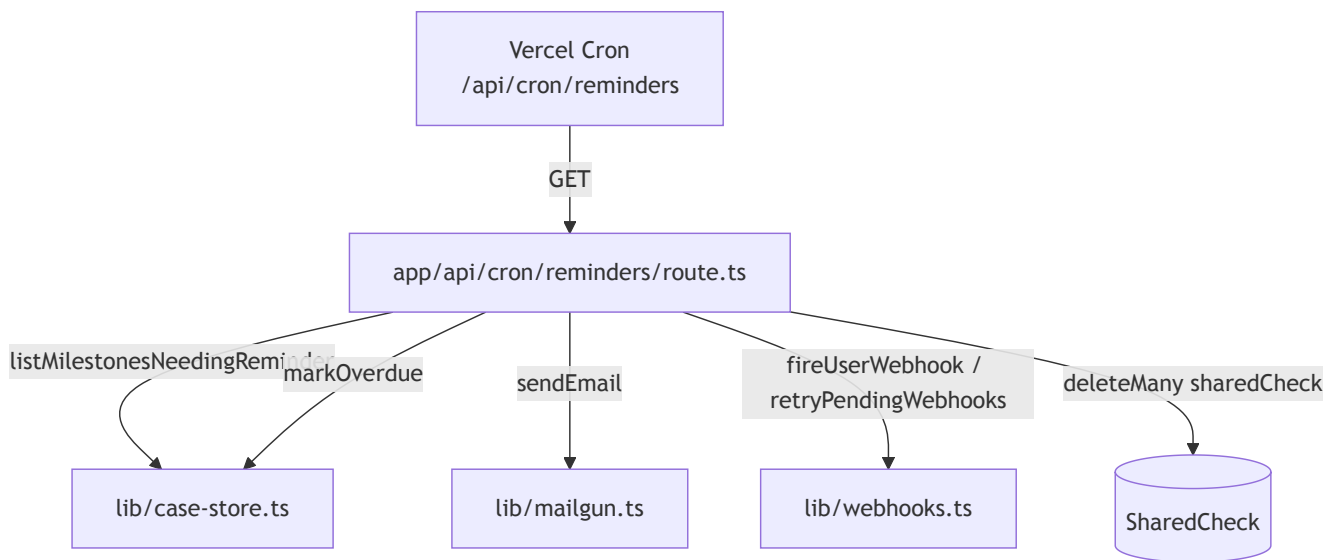
/check 會做規則式掃描並可選啟用 LLM 強化。 /api/check-text 具備 IP Rate Limit，超過上限會回傳 429。

4) 付款啟動到訂閱啟用



/checkout 建立 Order 後導向藍新付款。實際付款狀態以伺服器 notify 為準，成功時透過 activatePro 更新 BillingProfile。

5) 到期提醒 cron 與 webhook 重試



排程每日執行 D7/D1/D0/D+1 提醒，並對未過期未送出的 webhook 做重試，最後回傳本次發送/跳過結果。

「PAGES.md」

頁面 / 路由 / 模組清單

Web 路由（頁面）

路徑	用途	是否需登入
/	首頁、範本列表、導覽	否
/refund	退款頁（靜態）	否
/check	合約條文風險檢查頁	否
/en	英文首頁	否
/en/check	英文風險檢查頁	否
/disclaimer	免責聲明與律師轉介入口	否
/en/disclaimer	英文免責/律師轉介入口	否
/contracts	合約列表（依照 API 回傳顯示）	否（資料可透過 GET 取得）
/contracts/new	建立合約（範本填表）	否（會建立/讀取 docgen_uid）
/contracts/[id]	合約明細與里程碑	否（GET 無 owner filter）
/contracts/[id]/versions	修改紀錄與版本差異	需 uid，不符時回傳空清單
/contracts/[id]/sign	收件方簽署頁	不需登入，需 token
/cases	案件清單	否
/cases/[id]	案件細節/上傳附件	否
/templates/[id]	中文範本預覽/複製	否
/en/templates/[id]	英文範本預覽/複製	否
/admin	管理後台	否（需 ?key= 或 cookie）
/settings	帳號設定、Web API 金鑰、magic link	否（無 UID 則可讀取匿名 profile）
/terms	服務條款	否
/privacy	隱私權	否
/payment/success	付款完成結果頁	否
/shared/check/[id]	風險檢查分享頁	否

API 端點

方法	路徑	用途	存取規則
GET	/api/admin/overview	後台總覽	ADMIN_KEY 或授權 cookie (lib/admin-auth.ts)
GET	/api/admin/webhooks	後台 webhook 串流紀錄	管理者
POST	/api/admin/webhooks/retry	管理者手動重試 pending webhook	管理者
GET	/api/admin/contracts	後台合約清單	管理者
PATCH	/api/admin/contracts	變更合約 uid/caseId	管理者
DELETE	/api/admin/contracts	刪除合約 (可 force)	管理者
GET	/api/admin/export/{type}	匯出 CSV (contracts/orders/referrals/usage/deliveries)	管理者
GET	/api/admin/usage	用量與付費 KPI	管理者
GET	/api/admin/referrals	轉介案件清單	管理者
PATCH	/api/admin/referrals	更新轉介狀態/備註	管理者
GET	/api/auth/email/verify	magic link 驗證並綁定 UID	公開
POST	/api/auth/email/start	寄送 magic link (15 分鐘)	公開 (無登入需求)
GET	/api/billing/status	取得 UID/方案/用量	公開，必要時回寫 docgen_uid
GET	/api/billing/profile	讀取 BillingProfile	公開 (無 UID 時回寫匿名 UID)
PATCH	/api/billing/profile	更新 email / webhook 設定	需 docgen_uid
GET	/api/billing/api-keys	列出 API Key	需 docgen_uid
POST	/api/billing/api-keys	建立 API Key	需 docgen_uid
DELETE	/api/billing/api-keys	收回 API Key	需 docgen_uid
POST	/api/check-text	條文風險檢查	公開 + IP rate limit
POST	/api/risk-check	表單欄位風險檢查	公開
POST	/api/clause-suggest	AI 條款建議	公開
GET	/api/contracts	合約清單 (目前無 owner filter)	公開
POST	/api/contracts	建立合約	需 docgen_uid，必要時自動產生
GET	/api/contracts/[id]	查某一合約 (含里程碑)	公開
PATCH	/api/contracts/[id]	連結 case / 更新 expiryDate	目前無身份驗證
GET	/api/contracts/[id]/sign-fetch	依 token 讀取收件簽署資料	token 驗證
POST	/api/contracts/[id]/recipient-sign	收件方上傳簽名	token 驗證

方法	路徑	用途	存取規則
GET	/api/contracts/[id]/pdf	下載合約 PDF	token 驗證 + 簽署狀態檢查
GET	/api/contracts/[id]/versions	版本清單 (含當前值)	UID (不符不回錯, 回空)
PATCH	/api/contracts/[id]/values	送件人更新欄位、建立版本快照	sender uid 驗證
POST	/api/cases	建立案件	公開
GET	/api/cases	讀取案件列表	公開
GET	/api/cases/[id]	讀取單一案件	公開
PATCH	/api/cases/[id]	案件掛上 contractId	公開
POST	/api/cases/[id]/attachments	上傳案件附件到 Vercel Blob	公開
POST	/api/milestones	建立里程碑	公開
PATCH	/api/milestones/[id]	更新里程碑狀態	公開
GET	/api/milestones/[id]/dunning-prefill	根據到期/逾期生成催款預填值	公開
GET	/api/payment/status	查詢 merchantTradeNo 狀態	公開
POST	/api/payment/newwebpay/checkout	建立 NewwebPay 訂單參數	需 UID (無則自建)
POST	/api/payment/newwebpay/return	付費回傳頁面導向	由支付 callback 呼叫
POST	/api/payment/newwebpay/notify	支付結果最終授權	無條件 (第三方回呼)
POST	/api/referrals	提交律師轉介表單	公開
POST	/api/v1/contracts	對外合約 API (Rate Limit)	Authorization: Bearer dk_*
GET	/api/cron/reminders	每日提醒與 webhook 重試	CRON_SECRET (未設定時可直接叫)

註記

- app/api/cases、app/api/cases/[id]、/api/milestones、/api/contracts/[id] 目前皆未做完整使用者隔離；若要做多租戶，需補強 uid 權限控管。

「OPERATIONS.md」

安裝、執行、部署與維運

環境需求

- Node.js (建議使用 `package.json` 對應的現代版本)
- PostgreSQL 連線 (專案 README/DEPLOY 建議使用 Neon)
- 可選：NewwebPay、Mailgun、Vercel Blob、OpenRouter

安裝與本機啟動

```
npm install
npm run dev
```

`npm install` 會在 `postinstall` 觸發 `prisma generate`。

常用指令 (從 `package.json`)

- `npm run dev` : 啟動開發伺服器
- `npm run build` : `prisma generate` → `prisma db push --accept-data-loss --skip-generate` → `next build`
- `npm run start` : 啟動 production server
- `npm run lint` : ESLint
- `npm run test` : Vitest
- `npm run db:push` : 推送 schema 到資料庫
- `npm run db:migrate` : 執行 migrate (`prisma migrate deploy`)

環境變數

專案未附 `.env.example`，下列變數皆來自實際 `process.env` 取用。

分類	變數	是否必要	說明
資料庫	<code>DATABASE_POSTGRES_PRISMA_URL</code>	是	Prisma 主要連線 (pooled)
資料庫	<code>DATABASE_URL_UNPOOLED</code>	是	Prisma migrate 用 direct 連線
資料庫	<code>DATABASE_URL</code>	是 (多處邏輯判斷)	供連線可用性與 fallback 流程判斷
Blob 儲存	<code>BLOB_READ_WRITE_TOKEN</code>	否	簽名圖片儲存；未設定會 fallback 到 <code>public/signatures/</code> (dev)
NewwebPay	<code>NEWEBPAY_MERCHANT_ID</code>	否	支付店號
NewwebPay	<code>NEWEBPAY_HASH_KEY</code>	否	AES/TradeSha 金鑰

分類	變數	是否必要	說明
NewwebPay	NEWEBPAY_HASH_IV	否	AES/TradeSha IV
NewwebPay	NEWEBPAY_API_BASE	否	API base URL
Mailgun	MAILGUN_API_KEY	否	API Key
Mailgun	MAILGUN_DOMAIN	否	寄件 domain
Mailgun	MAILGUN_FROM	否	From 地址
Mailgun	MAILGUN_API_BASE	否	API endpoint
AI	OPENROUTER_API_KEY	否	/check-text 、 /clause-suggest 相關
AI	OPENROUTER_MODEL	否	可指定模型
AI	SMART_ROUTER_URL	否	lib/router-client.ts 路徑
站台/運行	NEXT_PUBLIC_APP_URL	否	簽署連結與驗證 redirect origin
站台/運行	NEXT_PUBLIC_SITE_URL	否	部分 API/頁面回呼 origin
Cron	CRON_SECRET	否	GET /api/cron/reminders 授權
安全	ADMIN_KEY	否 (建議)	/admin 與 admin API 認證
通知	ADMIN_ALERT_WEBHOOK	否	webhook 告警
通知	WEBHOOK_FAIL_ALERT_THRESHOLD	否	失敗告警門檻
轉介	REFERRAL_NOTIFY_TO	否	律師轉介通知信箱

所有變數僅列變數名，不附任何敏感值。

部署方式

Vercel (既有流程)

1. 連接 Neon PostgreSQL，設定 DATABASE_POSTGRES_PRISMA_URL 、 DATABASE_URL_UNPOOLED 、 DATABASE_URL 。
2. 設定 BLOB_READ_WRITE_TOKEN (若用 Vercel Blob) 。
3. 設定 NewwebPay/Mailgun 等服務變數。
4. 部署：

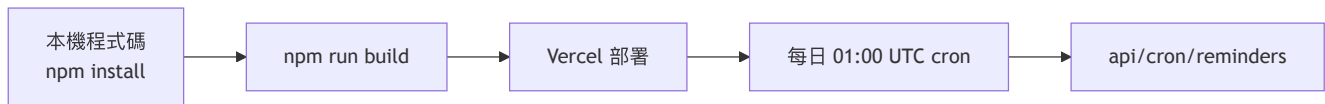
```
npm run build
vercel --prod
```

5. vercel.json 包含排程：

- GET /api/cron/reminders ，cron 0 1 * * *

其他

repo 未提供 Docker Compose 或 Makefile；請以 Node/Vercel 為主要部署路徑。



維運作業

- 移轉：
 - 開發期可用 `npm run db:push`
 - 生產版可用 `npm run db:migrate`
- 付款流程驗證：
 - 建立訂單：`POST /api/payment/newebpay/checkout`
 - 使用測試回報：由 `newebpay/notify` 導向實際 `Order.status`
- Webhook 管控：
 - 每日提醒流程同時呼叫 `retryPendingWebhooks`
 - 管理者可 `POST /api/admin/webhooks/retry` 立即重試
- 清理：
 - `api/cron/reminders` 會清掉到期的 `SharedCheck`

已確認腳本/文件

- 部署驗證與 smoke test 規格：`DEPLOY.md`
- 快速上手、API、資料模型、路由索引：本文檔與 `README.md`